Pilgrimage_2

Engineering Backlog and Implementation Guide

Lead engineer note. This document is intentionally more prescriptive than a normal brainstorm. Treat each story as a small ticket bundle: it states why the work exists, what files it touches, the concrete API shape, edge cases, and how to verify it. The main architectural rule is simple: one system owns each responsibility.

Priority	Story	Why now
P0	Stories 1-3	Harden card placement and board API before game rules depend on them.
P1	Stories 4-8	Add deck and action pipeline before effects/combat create coupling.
P2	Stories 9-11	Add effects after action flow exists.
P3	Stories 12-15	Build run setup, movement, combat, and end states.
P4	Stories 16-18	Create main scene/UI while preserving the test lab.

Current Design Constraints

- InputManager is currently the only drag/drop owner and should stay that way.
- Card uses data: CardData, health, attack. Do not reintroduce old names like cardData, cardHealth, or cardAttack.
- CardSlot already owns canAcceptCard(), setCard(), clearSlot(), and currentCard.
- SlotGrid currently generates the 3x3 slot array; it needs query helpers before movement/refill work.
- No hand system is planned. Journey deck plus board movement is the core play model.

Detailed Stories

Story 1 - Lock Down Card Placement Ownership

Goal. Make the current drag/drop path production-ready before adding new gameplay. A card should move from ON_BOARD to BEING_DRAGGED to either IN_SLOT or back to ON_BOARD through one obvious path.

Design justification. This preserves the rebuild rule: one owner per responsibility. InputManager owns pointer input, CardSlot owns whether a slot accepts a card, CardStateMachine owns state side effects. Do not recreate the old project's split input ownership.

Dependencies

- None. This is the foundation story.

Files to create/change

- src/singletons/input_manager.gd
- src/board/card_slots/card_slot.gd
- src/cards/card.gd
- src/cards/card_states/card_state_machine.gd

Implementation steps

1. Audit InputManager and keep all pointer drag/drop decisions there. It should connect to cardPressed, slotHovered, and slotUnhovered only.
2. Keep CardSlot.setCard(card) as the only direct slot placement method. InputManager may call it, but it should not reparent cards itself.
3. Make InputManager._dropCard(card) call card.cancelDrag() instead of repeating state/mouse_filter logic if you want to tighten encapsulation.
4. Remove remaining noisy prints from Card.onCardPressed() once card state tests are stable.
5. Do not add new board input scripts. Future board clicks should go through GlobalSignalBus and GameController, not a second drag manager.

Suggested API / shape

```
# Intended ownership
InputManager._finishDragging(mouse_pos)
-> targetSlot = _findDropTarget()
-> if targetSlot and targetSlot.canAcceptCard(card): targetSlot.setCard(card)
-> else: card.cancelDrag()

CardSlot.setCard(card)
-> currentCard = card
-> card.reparent(cardAnchor, false)
-> card.position = Vector2.ZERO
-> card.placeInSlot()
```

Edge cases

- Dropping onto an occupied slot must not overwrite currentCard.
- Dropping while the dragged card was freed must clear InputManager.cardBeingDragged safely.
- card.data may be null in bad test cases; CardSlot.canAcceptCard already guards this and should keep doing so.

Verification

- Spawn a card in card_test_scene, drag onto an empty slot, confirm currentState == IN_SLOT.
- Try to drag the slotted card again; canBeDragged() should reject it.
- Drag a second card onto the same slot; it should not replace the first card.

Story 2 - Add A Real SlotGrid Query API

Goal. Give the generated grid a reliable API for movement, refill, combat, and effects. Other systems should never crawl the GridContainer's children manually.

Design justification. The rebuild should prefer small, named APIs over reaching through scene trees. SlotGrid is the board's source of truth for coordinates and occupancy.

Dependencies

- Story 1

Files to create/change

- src/board/slot_grid/slot_grid.gd

Implementation steps

1. Rename class_name slot_grid to SlotGrid. Godot style and future type hints will be cleaner.
2. Change slot.coordinates assignment from Vector2(col, row) to Vector2i(col, row). CardSlot already exports Vector2i.
3. Decide gridSize meaning and document it. Current code treats gridSize.x as rows and gridSize.y as columns; either keep and name rows/columns or switch cleanly.
4. Add get_slot_at(coords: Vector2i) -> CardSlot using the slots[row][col] array.
5. Add get_empty_slots() -> Array[CardSlot] and get_occupied_slots() -> Array[CardSlot].
6. Add get_center_slot() -> CardSlot. For 3x3 this returns coordinates (1, 1).
7. Add get_cardinal_neighbours(slot: CardSlot) -> Array[CardSlot].

Suggested API / shape

```
func get_slot_at(coords: Vector2i) -> CardSlot:
if coords.x < 0 or coords.y < 0:
return null
if coords.y >= slots.size():
return null
if coords.x >= slots[coords.y].size():
return null
return slots[coords.y][coords.x]

func get_empty_slots() -> Array[CardSlot]:
var empty: Array[CardSlot] = []
for row in slots:
for slot in row:
if not slot.isOccupied():
empty.append(slot)
return empty
```

Edge cases

- If gridSize is Vector2i(0, 0), _createGrid() should create no slots but not crash.
- If slotScene is missing, push_error and return before instantiate().
- get_slot_at() must return null for out-of-bounds coordinates.

Verification

- In _ready() after _createGrid(), print/assert slots.size() == 3 for the current scene.
- get_center_slot().coordinates should be Vector2i(1, 1) for the current 3x3 grid.
- Corner slots should have two cardinal neighbours; center should have four.

Story 3 - Introduce BoardController As The Board Boundary

Goal. Create a small board-facing service that owns board operations: place, move, clear, query. It should not own input or combat.

Design justification. This stops future systems from calling CardSlot and SlotGrid internals directly. The board becomes a bounded subsystem rather than a collection of scene nodes everyone pokes.

Dependencies

- Story 2

Files to create/change

- src/board/board_controller.gd
- src/board/board_controller.tscn or main game scene

Implementation steps

1. Create BoardController extending Node or Control.
2. Export a NodePath to SlotGrid or assign it with @onready var grid: SlotGrid.
3. Add place_card(card: Card, slot: CardSlot) -> bool. It should call slot.setCard(card) after validation.
4. Add move_card(card: Card, target_slot: CardSlot) -> bool. It should clear the source slot, then place into the target.
5. Add clear_board() that loops grid.get_occupied_slots() and calls clearSlot().
6. Emit a new GlobalSignalBus.boardStateChanged signal after successful place/move/clear.

Suggested API / shape

```
class_name BoardController
extends Node

@export var grid_path: NodePath
@onready var grid: SlotGrid = get_node(grid_path)

func place_card(card: Card, slot: CardSlot) -> bool:
if slot == null or not slot.canAcceptCard(card):
return false
var ok := slot.setCard(card)
if ok:
GlobalSignalBus.emitBoardStateChanged()
return ok
```

Edge cases

- Moving from a slot should clear the old slot's currentCard.
- Moving to an occupied slot should return false; combat will handle occupied targets later.
- BoardController should not call InputManager.lockInput except during future scripted animations.

Verification

- Call BoardController.place_card from a test button and confirm slotFilled plus boardStateChanged fire.
- Call clear_board and confirm every currentCard is null.
- No drag code appears in BoardController.

Story 4 - Build DeckModel Around Card Ids

Goal. Add a generic deck that stores card ids, not Card nodes. It can shuffle, draw, add to top/bottom, and report count.

Design justification. Cards are expensive scene instances and should be created only when revealed. Data-driven ids keep the deck simple, serializable, and testable.

Dependencies

- CardLibrary autoload already exists

Files to create/change

- src/decks/deck_model.gd
- src/decks/deck_model.tscn optional

Implementation steps

1. Create DeckModel with class_name DeckModel.
2. Use var cards: Array[String] = [] rather than untyped arrays.
3. Add initialise_deck(card_ids: Array[String]) that duplicates input and shuffles optionally.
4. Add draw_card_id() -> String that pop_fronts safely and returns an empty string when empty.
5. Add draw_card() -> Card that calls CreateCard.createCard(id), but keep draw_card_id available for tests.
6. Add get_deck_size(), add_card_to_top(id), add_card_to_bottom(id), shuffle_deck().
7. Add deck signals to GlobalSignalBus: deckShuffled(deck), deckEmptied(deck), cardDrawn(card).

Suggested API / shape

```
func draw_card() -> Card:
var card_id := draw_card_id()
if card_id == "":
return null
var card := CreateCard.new().createCard(card_id)
if card != null:
GlobalSignalBus.emitCardDrawn(card)
return card
```

Edge cases

- CreateCard is currently not an autoload. Either instantiate CreateCard.new() or promote it later.
- If CardLibrary does not have an id, CreateCard returns null; deck should handle that without losing game flow.
- Do not put deck click UI behavior in DeckModel yet.

Verification

- Initialize with three ids; get_deck_size returns 3.
- Draw three cards; size reaches 0 and fourth draw returns null.
- Shuffle does not change count.

Story 5 - Implement JourneyDeck Board Refill

Goal. Add the board-adventure draw pile. It reveals cards into empty slots and refills the previous slot after the player moves.

Design justification. This supports the no-hand design: the board is the game space, and the journey deck is how the game introduces choices.

Dependencies

- Story 2
- Story 3
- Story 4

Files to create/change

- src/decks/journey_deck.gd

- src/decks/journey_deck.tscn optional

Implementation steps

1. Create JourneyDeck extending DeckModel.
2. Add initialise_journey_deck() with a temporary preset list using ids from data/card_dictionary.json.
3. Add reveal_top_card(slot: CardSlot) -> Card. It draws a card, calls BoardController.place_card or slot.setCard, and flips/reveals if needed.
4. Add fill_empty_slots(grid: SlotGrid) to iterate grid.get_empty_slots().
5. Later, route reveal_top_card through ActionQueue using REVEAL_CARD instead of direct placement.

Suggested API / shape

```
func fill_empty_slots(grid: SlotGrid) -> void:
for slot in grid.get_empty_slots():
if get_deck_size() <= 0:
return
reveal_top_card(slot)
```

Edge cases

- Do not fill the center slot after the player is placed there.
- If deck empties mid-fill, stop without error.
- If CreateCard returns null for a bad id, continue to the next slot or fail loudly during setup.

Verification

- With a 3x3 empty grid and at least 9 ids, fill_empty_slots fills 9 slots.
- With player in center first, fill_empty_slots fills 8 slots.
- Deck count decreases by the number of revealed cards.

Story 6 - Define Actions As Gameplay Contracts

Goal. Create named action dictionaries before adding effects and combat. Actions describe intent; processors apply changes.

Design justification. This is the key rebuild improvement over direct script-to-script mutation. It makes effects possible without spaghetti.

Dependencies

- Stories 1-5

Files to create/change

- src/actions/action_types.gd

Implementation steps

1. Create class_name ActionTypes.
2. Define const REVEAL_CARD, MOVE_CARD, MODIFY_STATS, DESTROY_CARD, DEAL_DAMAGE, DRAW_CARD, GAME_OVER.
3. Add static make(type: String, source=null, target=null, data: Dictionary={}) -> Dictionary.
4. Add static is_valid(action: Dictionary) -> bool.
5. Do not put action execution here.

Suggested API / shape

```
static func make(type: String, source = null, target = null, data: Dictionary = {}) ->
Dictionary:
return {
"type": type,
"source": source,
"target": target,
"data": data
}
```

Edge cases

- Use constants everywhere. Do not type raw strings like 'destroy_card' in effect scripts.
- Keep action data dictionaries small and documented per action.

Verification

- ActionTypes.is_valid(ActionTypes.make(ActionTypes.MOVE_CARD)) returns true.
- Unknown type returns false or warns.

Story 7 - Add ActionQueue Autoload

Goal. Add a FIFO queue so gameplay state changes run in a predictable order.

Design justification. Effects, combat, and deck refill need a shared timeline. A queue prevents nested direct calls where one system mutates state during another system's work.

Dependencies

- Story 6

Files to create/change

- src/singletons/actions/action_queue.gd
- project.godot autoload

Implementation steps

1. Create action_queue.gd extending Node.
2. Add var _queue: Array[Dictionary] = [].
3. Add enqueue_action(action), pop_next_action(), peek_next_action(), queue_has_actions(), clear_queue(), get_queue_size().
4. Validate actions using ActionTypes.is_valid before appending.
5. Add actionEnqueued, actionPopped, queueCleared signals locally or via GlobalSignalBus.
6. Register ActionQueue as an autoload in project.godot.

Suggested API / shape

```
func enqueue_action(action: Dictionary) -> void:
if not ActionTypes.is_valid(action):
push_warning("ActionQueue rejected invalid action: %s" % [action])
return
_queue.append(action)
emit_signal("actionEnqueued", action)
```

Edge cases

- Never process actions inside enqueue_action.
- Avoid queueing empty dictionaries.
- Clear queue on new run setup to avoid old actions firing in a new game.

Verification

- Queue size increments after valid enqueue.
- Invalid action does not change queue size.
- pop_next_action returns actions in insertion order.

Story 8 - Add ActionProcessor Autoload

Goal. Resolve queued actions from one place. This becomes the gate for board changes, stat changes, destruction, and later effects.

Design justification. This keeps rules deterministic. Systems request changes; ActionProcessor performs changes. That is the lead-engineer boundary that prevents the rebuild becoming the old project again.

Dependencies

- Story 6
- Story 7
- Story 3

Files to create/change

- src/singletons/actions/action_processor.gd
- project.godot autoload

Implementation steps

1. Create action_processor.gd extending Node.
2. In _process, if not busy and ActionQueue has actions, pop one and resolve it.
3. Before resolving, call EffectMediator.on_action_pre(action) once that exists.
4. After resolving, call EffectMediator.on_action_post(action).
5. Implement handlers: _handle_reveal_card, _handle_move_card, _handle_modify_stats, _handle_destroy_card, _handle_deal_damage.
6. Keep visual animation minimal until logic is stable.

Suggested API / shape

```
func _resolve_action(action: Dictionary) -> void:
match str(action.get("type", "")):
ActionTypes.REVEAL_CARD:
_handle_reveal_card(action)
ActionTypes.MOVE_CARD:
_handle_move_card(action)
ActionTypes.MODIFY_STATS:
_handle_modify_stats(action)
_:
push_warning("Unknown action: %s" % [action])
```

Edge cases

- Destroyed cards must clear their slot before queue_free.
- MODIFY_STATS should clamp health if you decide health cannot go below 0.
- MOVE_CARD to an occupied slot should fail unless combat has already cleared it.

Verification

- Enqueue MODIFY_STATS and confirm card visuals update.
- Enqueue DESTROY_CARD and confirm slot currentCard is null.
- Processor does not re-enter itself while busy.

Story 9 - Add Effect Data Registry

Goal. Load data/effect_dictionary.json into typed EffectData resources and make them available by id.

Design justification. This mirrors CardData and keeps effects data-driven. Cards stay declarative: they list effect ids; runtime systems decide what those ids do.

Dependencies

- Card data loader pattern exists

Files to create/change

- src/effects/effect_data.gd
- src/effects/effect_data_factory.gd
- src/singletons/registries/effect_data_registry.gd
- data/effect_dictionary.json

Implementation steps

1. Create EffectData resource with id, trigger, effectType, effectParameters.
2. Create EffectDataFactory.fromDictionary(dictionary).
3. Create EffectDictionaryJsonLoader or reuse a generic JSON loader if you extract one.
4. Create EffectDataRegistry autoload with get_effect_data(effect_id).
5. Load all effects at _ready and store by id.

Suggested API / shape

```
class_name EffectData
extends Resource

@export var id: String
@export var trigger: String
@export var effectType: String
@export var effectParameters: Dictionary = {}
```

Edge cases

- Missing effects referenced by CardData.effects should warn but not crash card creation.
- Effect parameter names should match JSON exactly at the boundary, then convert to camelCase inside typed code if desired.

Verification

- Every id in data/effect_dictionary.json can be fetched from EffectDataRegistry.
- Bad id returns null and logs a useful warning.

Story 10 - Add EffectMediator And Effect Instances

Goal. Create runtime effect objects from EffectData and dispatch action hooks to them.

Design justification. This keeps effects from directly subscribing all over the project. The mediator owns listeners; effects own their own reaction logic.

Dependencies

- Story 6
- Story 7
- Story 8
- Story 9

Files to create/change

- src/effects/card_effect_factory.gd
- src/singletons/effects/effect_mediator.gd
- src/cards/card.gd or CardSlot.setCard cleanup hook

Implementation steps

1. Create CardEffectFactory.create(card: Card, effect_data: EffectData).
2. Create EffectMediator autoload with listeners: Array[Dictionary].
3. Add register_card_effects(card) that loops card.data.effects, creates effect instances, and stores {card, effect, trigger}.
4. Call register_card_effects when a card enters IN_SLOT or after CardSlot.setCard succeeds.
5. Add remove_listeners_for_card(card) and call it from CardSlot.clearSlot and destroy actions.
6. Add on_action_pre(action) and on_action_post(action), called by ActionProcessor.

Suggested API / shape

```
func on_action_post(action: Dictionary) -> void:
_clean_dead_listeners()
for listener in listeners:
var effect = listener.get("effect")
if effect != null and effect.has_method("on_action"):
effect.on_action(action, "post")
```

Edge cases

- Avoid registering the same card's effects twice if setCard is called repeatedly.
- Clean up listeners when cards are queue_freed.
- Effects should enqueue actions, not directly alter other cards.

Verification

- Place a card with an effect id and confirm mediator listener count increases.
- Clear/destroy the card and confirm listener count decreases.
- Post-action dispatch calls effect.on_action.

Story 11 - Implement The First Three Effects

Goal. Build SolitaryBeast, HealOnKill, and BuffAttackOnKill as proof that the action/effect architecture works.

Design justification. This deliberately tests three important effect classes: board-state recalculation, pre-destroy reaction, and stat modification.

Dependencies

- Story 10

Files to create/change

- src/effects/card_effects/solitary_beast.gd
- src/effects/card_effects/heal_on_kill.gd
- src/effects/card_effects/buff_attack_on_kill.gd

Implementation steps

1. SolitaryBeast: on board-changing post actions, count matching cards on the board and enqueue MODIFY_STATS for host card.
2. HealOnKill: on DESTROY_CARD pre action, if host card is the destroyed target, heal the destroyer.

3. BuffAttackOnKill: on DESTROY_CARD pre action, if host card is the destroyed target, increase destroyer's attack.
4. Read base stats from card.data.baseHealth/baseAttack and current stats from card.health/card.attack.
5. Do not use old card.cardHealth/card.cardAttack names.

Suggested API / shape

```
# Example effect rule
func on_action(action: Dictionary, when: String) -> void:
if when != "pre":
return
if action.get("type") != ActionTypes.DESTROY_CARD:
return
var attacker: Card = action.get("source")
var destroyed: Card = action.get("target")
if destroyed != hostCard:
return
ActionQueue.enqueue_action(ActionTypes.make(ActionTypes.MODIFY_STATS, hostCard, attacker,
{"health": attacker.health + hostCard.data.baseHealth}))
```

Edge cases

- Prevent infinite loops: SolitaryBeast should react to board-changing actions, not its own MODIFY_STATS action unless intentional.
- If attacker is null or freed, effect should return.
- If hostCard is freed, effect should return and mediator should clean it later.

Verification

- A Woods card entering board changes SolitaryBeast stats.
- Destroying a HealOnKill host increases attacker health.
- Destroying a BuffAttackOnKill host increases attacker attack.

Story 12 - Add GameController Setup

Goal. Start a run: create player, place them in the center, initialize journey deck, fill remaining slots, enter PLAYER_TURN.

Design justification. Setup belongs in one run controller, not in card tests or deck scripts. This creates a real game spine while keeping UI optional.

Dependencies

- Stories 2-5

Files to create/change

- src/singletons/managers/game_controller.gd or src/game/game_controller.gd
- main.tscn or new src/main/game_scene.tscn

Implementation steps

1. Create GameController with enum GameState { SETUP, PLAYER_TURN, MOVING, COMBAT, GAME_OVER }.
2. Export NodePaths for BoardController, SlotGrid, JourneyDeck.
3. On start_run(), lock InputManager, clear board, create player with CreateCard.
4. Place player in slot_grid.get_center_slot().
5. Initialize JourneyDeck and call fill_empty_slots.
6. Unlock InputManager and set state PLAYER_TURN.

Suggested API / shape

```
func start_run() -> void:
current_state = GameState.SETUP
InputManager.lockInput()
board.clear_board()
var player := CreateCard.new().createCard("C_0000")
board.place_card(player, grid.get_center_slot())
journey_deck.initialise_journey_deck()
journey_deck.fill_empty_slots(grid)
current_state = GameState.PLAYER_TURN
InputManager.unlockInput()
```

Edge cases

- If player card creation fails, stay in SETUP and push_error.
- If center slot is missing, fail setup loudly.
- Do not let journey deck overwrite center slot.

Verification

- Running main scene places one player in center.
- Other empty slots fill from journey deck.
- Game state becomes PLAYER_TURN only after setup completes.

Story 13 - Add Board Movement Rules

Goal. Allow the player character to move to cardinal-adjacent slots and reject illegal movement.

Design justification. The board is the play space. Movement rules should be explicit and readable rather than hidden in input code.

Dependencies

- Story 12

Files to create/change

- src/singletons/managers/game_controller.gd
- src/board/slot_grid/slot_grid.gd

Implementation steps

1. Add a way to identify the player card: CardData.isPlayer already exists.
2. On slotClicked or cardPressed for board cards, resolve target slot.
3. Find the current player slot using grid.get_occupied_slots() and card.data.isPlayer.
4. Check target slot is in grid.get_cardinal_neighbours(player_slot).
5. If target is empty, enqueue MOVE_CARD.
6. After move succeeds, enqueue/refill previous slot from JourneyDeck.

Suggested API / shape

```
func is_valid_move(from_slot: CardSlot, to_slot: CardSlot) -> bool:
if from_slot == null or to_slot == null:
return false
var delta := to_slot.coordinates - from_slot.coordinates
return abs(delta.x) + abs(delta.y) == 1
```

Edge cases

- Clicking the player card itself should not move.
- Clicking diagonal slots should do nothing or emit invalid feedback.

- During COMBAT or GAME_OVER, ignore movement input.

Verification

- Center to top/left/right/bottom works.
- Center to corner is rejected.
- Previous slot refills after a successful empty-slot move.

Story 14 - Add Combat Resolution

Goal. When the player moves into an occupied adjacent slot, resolve combat instead of simple movement.

Design justification. Combat is game rule logic. It should produce actions and signals, allowing effects and UI to react without special casing.

Dependencies

- Story 8
- Story 13

Files to create/change

- src/game/combat_resolver.gd or GameController method
- src/actions/action_types.gd
- src/singletons/actions/action_processor.gd

Implementation steps

1. Add CombatResolver.calculate(attacker: Card, defender: Card) -> Dictionary or a GameController method.
2. Define whether attack > health or attack >= health kills. Old code used >; choose and document.
3. Enqueue DEAL_DAMAGE for defender and attacker as needed.
4. If defender dies, enqueue DESTROY_CARD with source attacker target defender.
5. If attacker survives and defender is destroyed, enqueue MOVE_CARD into defender slot.
6. Emit battleCompleted(attacker, defender, result) through GlobalSignalBus.

Suggested API / shape

```
var result := {
"attacker_survives": attacker.health > defender.attack,
"defender_destroyed": attacker.attack >= defender.health,
"attacker_damage": defender.attack,
"defender_damage": attacker.attack
}
```

Edge cases

- Buff cards with 0 attack may be consumed without damaging player if that is intended.
- If attacker dies, do not move into defender slot.
- Effects triggered by DESTROY_CARD must run before destroyed card is freed.

Verification

- Attacking weak enemy destroys it and moves player.
- Attacking strong enemy damages or kills player.
- battleCompleted signal includes enough data for UI.

Story 15 - Add Game Over State

Goal. End the run cleanly when the player dies or another terminal condition occurs.

Design justification. Clear state boundaries prevent lingering input, queued actions, and invalid board interactions after the run should be over.

Dependencies

- Story 14

Files to create/change

- src/singletons/managers/game_controller.gd
- src/singletons/global_signal_bus.gd

Implementation steps

1. Add signal gameOver(reason) and emitGameOver(reason) wrapper to GlobalSignalBus.
2. In GameController, add enter_game_over(reason: String).
3. Call InputManager.lockInput() and ActionQueue.clear_queue() when entering GAME_OVER.
4. Trigger game over when player health <= 0 after combat/action processing.
5. Decide and document empty journey deck outcome: win, exhaustion, or continue until board cleared.

Suggested API / shape

```
func enter_game_over(reason: String) -> void:
current_state = GameState.GAME_OVER
InputManager.lockInput()
ActionQueue.clear_queue()
GlobalSignalBus.emitGameOver(reason)
```

Edge cases

- Do not emit gameOver more than once.
- Queued actions from the previous state should not continue after GAME_OVER unless explicitly allowed.

Verification

- Player death locks input.
- UI/debug receives a gameOver reason.
- Trying to move after game over does nothing.

Story 16 - Build A Real Game Scene

Goal. Create a playable scene separate from card_test_scene. The test scene remains a lab; the main scene owns the real run.

Design justification. The old project let test and production ideas blur. The rebuild should separate experiments from the scene the player actually runs.

Dependencies

- Stories 3-5
- Story 12

Files to create/change

- src/main/game_scene.tscn
- main.tscn
- project.godot

Implementation steps

1. Create a new game scene with SlotGrid, BoardController, JourneyDeck, and GameController.
2. Wire exported NodePaths in the editor rather than hardcoding long get_node paths.
3. Keep card_test_scene.tscn as project main until game_scene setup is reliable.
4. Once stable, update project.godot run/main_scene to the new scene.
5. Remove test-only buttons from the real game scene.

Suggested API / shape

```
GameScene
CanvasLayer/UI
Board
SlotGrid
BoardController
Decks
JourneyDeck
GameController
```

Edge cases

- Do not instance BoardInputControler again; it was intentionally removed.
- Do not depend on card_test_scene.gd constants like CARD_START_POS.

Verification

- Opening game_scene starts a run without using test buttons.
- card_test_scene still runs for card experiments.
- No missing NodePath errors appear in output.

Story 17 - Minimal Run UI

Goal. Show just enough UI to make the run understandable: state, deck count, combat feedback, game over.

Design justification. This supports the no-hand design. UI should inform the board game, not become a second card system.

Dependencies

- Story 16

Files to create/change

- src/ui/run_status.gd
- src/ui/run_status.tscn
- src/singletons/global_signal_bus.gd

Implementation steps

1. Create a small Control scene anchored to a corner.
2. Show current GameController state.
3. Show JourneyDeck count after draw/reveal.
4. Listen to battleCompleted and show short feedback text.
5. Listen to gameOver and show final reason.
6. Keep it read-only; no buttons except perhaps restart later.

Suggested API / shape

```
RunStatus listens to:
- boardStateChanged
- battleCompleted(attacker, defender, result)
- gameOver(reason)
- deck count changes
```

Edge cases

- UI should not own game rules.
- If a signal arrives before references are ready, ignore safely.

Verification

- Deck count changes when slots are filled.
- Combat message appears after fight.
- Game over message appears and remains visible.

Story 18 - Keep Card Test Scene As A Lab

Goal. Protect the card test scene as a place for experiments while preventing it from defining production architecture.

Design justification. This gives you speed without mess. Experiments stay in src/tests; production code remains clear.

Dependencies

- Always ongoing

Files to create/change

- src/tests/card_test_scene.tscn
- src/tests/card_test_scene.gd

Implementation steps

1. Keep spawn buttons for representative cards.
2. Keep state/stat debug controls only in the test scene.
3. Add future buttons for effect test cards after effects are implemented.
4. Remove or gate noisy debug printing.
5. Never make GameController depend on card_test_scene.gd.

Suggested API / shape

```
Allowed in tests:
- manual card spawning
- stat cycling
- effect trigger probes
- visual sizing experiments

Not allowed in production:
- test buttons
- debug-only layout constants
- assumptions about CardsRoot
```

Edge cases

- Test scene may be messy, but mess should not leak into src/main or src/singletons.
- If a test helper becomes generally useful, extract it into a real helper with a clear name.

Verification

- Card test still creates cards after main scene exists.
- Main scene contains no test controls.
- Debug print noise is limited to active investigations.

Definition Of Done For This Backlog

- A new developer can open a story and know which files to touch without asking for architectural context.
- Every new singleton has a clear reason to be global and is listed in project.godot.
- Every gameplay state mutation either belongs to CardStateMachine, CardSlot/BoardController, or ActionProcessor.
- The card test scene remains useful but is not required for the main game scene.
- Effects do not directly mutate unrelated cards; they enqueue actions.